

Informe de Preparación de la Base de Datos Espacial para el Proyecto LADM_COL SINIC

Diego Culma Estepa - Proyecto Final BDE
Ingeniería Catastral y Geodesia

6 de junio de 2026

1. Introducción

El presente informe describe el proceso de preparación inicial de la base de datos espacial para el desarrollo del proyecto final de Bases de Datos Espaciales, orientado a la estructuración de datos catastrales conforme al Modelo de Aplicación LADM_COL SINIC V1.0.

La base de datos constituye el componente central para la gestión, almacenamiento, consulta y actualización de la información catastral que posteriormente será consumida por una aplicación web compuesta por un *backend* y un *frontend*. En esta primera fase se prioriza la creación de una estructura funcional, organizada y técnicamente coherente con los componentes mínimos exigidos en el proyecto: interesados, unidades administrativas, unidades espaciales, topografía y representación, y cartografía catastral.

Aunque el modelo LADM_COL SINIC completo posee una estructura amplia y especializada, para el alcance académico del proyecto se plantea una versión simplificada, enfocada en permitir operaciones CRUD, manejo de geometrías espaciales y trazabilidad básica mediante campos de vigencia temporal y auditoría.

2. Objetivo de la preparación de la base de datos

Preparar una base de datos espacial en PostgreSQL con extensión PostGIS, organizada mediante esquemas y tablas principales, que permita soportar el desarrollo posterior de una aplicación web para la gestión básica de datos catastrales conforme al enfoque conceptual del Modelo de Aplicación LADM_COL SINIC V1.0.

3. Estructura general del proyecto

Para mantener una organización clara entre los distintos componentes de la aplicación, se definió una estructura de carpetas separada para la base de datos, el backend y el frontend:

```
proyecto-ladm-sinic/  
|-- backend/  
|-- frontend/  
|-- database/  
|   |-- 01_create_database.sql  
|   |-- 02_create_tables.sql  
|   |-- 03_seed_data.sql  
|-- README.md
```

Esta organización permite separar responsabilidades. La carpeta **database** contiene los scripts SQL necesarios para crear y poblar la base de datos; la carpeta **backend** contendrá la API encargada de comunicarse con PostgreSQL; y la carpeta **frontend** contendrá la interfaz web mediante la cual el usuario podrá consultar, crear, actualizar y eliminar registros.

4. Creación de la base de datos

La base de datos propuesta se denomina:

`bde_ladm_sinic`

Este nombre permite identificar claramente que se trata de una base de datos espacial asociada al proyecto LADM_COL SINIC.

La creación puede realizarse desde pgAdmin o mediante consola. Desde SQL, la instrucción base sería:

```
1 CREATE DATABASE bde_ladm_sinic;
```

Listing 1: Creación de la base de datos

Una vez creada la base de datos, se debe abrir el *Query Tool* sobre ella para ejecutar las instrucciones de configuración inicial.

5. Extensiones utilizadas

Para este proyecto se decidió utilizar únicamente las extensiones necesarias para el manejo espacial y la generación de identificadores únicos. No se emplea la extensión `postgis_topology`, debido a que el alcance del proyecto no requiere manejo topológico avanzado.

Las extensiones activadas son:

```
1 CREATE EXTENSION IF NOT EXISTS postgis;  
2 CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Listing 2: Activación de extensiones

La extensión `postgis` permite almacenar y consultar geometrías espaciales dentro de PostgreSQL. Por su parte, `pgcrypto` permite generar identificadores UUID mediante la función `gen_random_uuid()`, lo cual resulta adecuado para manejar claves primarias únicas en las tablas principales.

6. Organización mediante esquemas

Para evitar una estructura desordenada dentro de la base de datos, se definieron tres esquemas principales:

Esquema	Función
ladm_sinic	Contiene las tablas principales del modelo catastral.
catalogos	Permite almacenar listas controladas o dominios de valores.
auditoria	Contiene funciones y mecanismos de control de actualización.

Cuadro 1: Esquemas definidos para la base de datos

La creación de los esquemas se realiza mediante:

```
1 CREATE SCHEMA IF NOT EXISTS ladm_sinic;  
2 CREATE SCHEMA IF NOT EXISTS catalogos;  
3 CREATE SCHEMA IF NOT EXISTS auditoria;
```

Listing 3: Creación de esquemas

Esta separación facilita el mantenimiento de la base de datos y permite que el backend acceda de manera ordenada a las tablas del modelo.

7. Usuario de conexión para el backend

Como buena práctica, se propuso no conectar el backend directamente con el usuario administrador `postgres`. En su lugar, se crea un usuario específico para la aplicación:

```
1 CREATE USER sinic_app WITH PASSWORD 'sinic_app_123';
2
3 GRANT CONNECT ON DATABASE bde_ladm_sinic TO sinic_app;
```

Listing 4: Creación del usuario para el backend

Posteriormente, se otorgan permisos sobre los esquemas que serán utilizados por la aplicación:

```
1 GRANT USAGE ON SCHEMA ladm_sinic TO sinic_app;
2 GRANT USAGE ON SCHEMA catalogos TO sinic_app;
3 GRANT USAGE ON SCHEMA auditoria TO sinic_app;
4
5 GRANT CREATE ON SCHEMA ladm_sinic TO sinic_app;
6 GRANT CREATE ON SCHEMA catalogos TO sinic_app;
```

Listing 5: Permisos sobre esquemas

Esta decisión mejora la seguridad y permite separar las tareas de administración de la base de datos de las operaciones propias de la aplicación web.

8. Sistema de referencia espacial

Para las geometrías del proyecto se definió el sistema de referencia EPSG:9377, correspondiente a MAGNA-SIRGAS / Origen Nacional. Este sistema resulta adecuado para el manejo de información geográfica catastral en Colombia, especialmente cuando se busca trabajar con coordenadas proyectadas y mediciones en unidades métricas.

La verificación del sistema de referencia puede realizarse mediante la siguiente consulta:

```
1 SELECT srid, auth_name, auth_srid, srtext
2 FROM spatial_ref_sys
3 WHERE srid = 9377;
```

Listing 6: Verificación del SRID 9377

9. Tablas principales del modelo

La estructura inicial de la base de datos se diseñó a partir de cinco componentes principales exigidos para los CRUD del proyecto: interesados, unidad administrativa, unidad espacial, topografía y representación, y cartografía catastral. Adicionalmente, se incluye una tabla intermedia para relacionar interesados con unidades administrativas mediante derechos.

9.1. Tabla de interesados

La tabla `interesado` almacena la información básica de las personas naturales, jurídicas o entidades que tienen relación con una unidad administrativa. Puede representar propietarios, poseedores, ocupantes u otros actores vinculados al predio.

```
1 CREATE TABLE ladm_sinic.interesado (  
2     id_interesado UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     tipo_interesado VARCHAR(50) NOT NULL,  
4     tipo_documento VARCHAR(50),  
5     numero_documento VARCHAR(50) UNIQUE,  
6     nombre_razon_social VARCHAR(255) NOT NULL,  
7     correo VARCHAR(150),  
8     telefono VARCHAR(50),  
9     direccion TEXT,  
10    fecha_creacion TIMESTAMP DEFAULT NOW(),  
11    fecha_actualizacion TIMESTAMP DEFAULT NOW()  
12 );
```

Listing 7: Tabla de interesados

9.2. Tabla de unidad administrativa

La tabla `unidad_administrativa` representa el componente administrativo y jurídico del predio. Contiene información como código predial, matrícula inmobiliaria, municipio, dirección, destino económico, áreas y avalúo catastral.

```
1 CREATE TABLE ladm_sinic.unidad_administrativa (  
2     id_unidad_administrativa UUID PRIMARY KEY DEFAULT gen_random_uuid  
3     ),  
4     codigo_predial VARCHAR(50) UNIQUE NOT NULL,  
5     matricula_inmobiliaria VARCHAR(50),  
6     departamento VARCHAR(100),  
7     municipio VARCHAR(100),  
8     direccion TEXT,  
9     tipo_predio VARCHAR(100),  
10    destinacion_economica VARCHAR(100),  
11    area_terreno NUMERIC(18,2),  
12    area_construida NUMERIC(18,2),  
13    avaluo_catastral NUMERIC(18,2),  
14    fecha_inicio_vigencia DATE DEFAULT CURRENT_DATE,  
15    fecha_fin_vigencia DATE,  
16    estado VARCHAR(30) DEFAULT 'activo',  
17    fecha_creacion TIMESTAMP DEFAULT NOW(),  
18    fecha_actualizacion TIMESTAMP DEFAULT NOW()  
);
```

Listing 8: Tabla de unidad administrativa

9.3. Tabla de relación derecho-interesado

La tabla `derecho_interesado` permite asociar uno o varios interesados con una unidad administrativa. Esta tabla es necesaria porque una unidad administrativa puede tener varios interesados y un interesado puede estar vinculado a varios predios.

```
1 CREATE TABLE ladm_sinic.derecho_interesado (  
2     id_derecho UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     id_interesado UUID NOT NULL,  
4     id_unidad_administrativa UUID NOT NULL,  
5     tipo_derecho VARCHAR(100) NOT NULL,  
6     porcentaje_participacion NUMERIC(5,2),  
7     observacion TEXT,  
8     fecha_inicio_vigencia DATE DEFAULT CURRENT_DATE,  
9     fecha_fin_vigencia DATE,  
10    estado VARCHAR(30) DEFAULT 'activo',  
11  
12    CONSTRAINT fk_derecho_interesado  
13        FOREIGN KEY (id_interesado)  
14        REFERENCES ladm_sinic.interesado(id_interesado)  
15        ON DELETE CASCADE,  
16  
17    CONSTRAINT fk_derecho_unidad_administrativa  
18        FOREIGN KEY (id_unidad_administrativa)  
19        REFERENCES ladm_sinic.unidad_administrativa(  
20            id_unidad_administrativa)  
21        ON DELETE CASCADE  
22 );
```

Listing 9: Tabla de derechos e interesados

9.4. Tabla de unidad espacial

La tabla `unidad_espacial` representa la dimensión geométrica del predio. Incluye una geometría tipo polígono asociada a una unidad administrativa. Esta tabla permite almacenar el componente espacial del catastro y realizar consultas geográficas.

```
1 CREATE TABLE ladm_sinic.unidad_espacial (  
2     id_unidad_espacial UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     id_unidad_administrativa UUID NOT NULL,  
4     tipo_unidad_espacial VARCHAR(100) NOT NULL,  
5     etiqueta VARCHAR(100),  
6     area_calculada NUMERIC(18,2),  
7     geometria GEOMETRY(POLYGON, 9377) NOT NULL,  
8     fecha_inicio_vigencia DATE DEFAULT CURRENT_DATE,  
9     fecha_fin_vigencia DATE,  
10    estado VARCHAR(30) DEFAULT 'activo',  
11    fecha_creacion TIMESTAMP DEFAULT NOW(),  
12    fecha_actualizacion TIMESTAMP DEFAULT NOW(),  
13  
14    CONSTRAINT fk_unidad_espacial_administrativa  
15        FOREIGN KEY (id_unidad_administrativa)  
16        REFERENCES ladm_sinic.unidad_administrativa(  
17            id_unidad_administrativa)  
18        ON DELETE CASCADE  
19    );
```

Listing 10: Tabla de unidad espacial

Para optimizar las consultas espaciales, se crea un índice GIST sobre el campo geométrico:

```
1 CREATE INDEX idx_unidad_espacial_geom  
2 ON ladm_sinic.unidad_espacial  
3 USING GIST (geometria);
```

Listing 11: Índice espacial para unidad espacial

9.5. Tabla de topografía y representación

La tabla `topografia_representacion` almacena elementos levantados o representados espacialmente, como vértices, puntos de control, mojones, linderos u otros elementos relacionados con la representación física del predio.

```
1 CREATE TABLE ladm_sinic.topografia_representacion (  
2     id_topografia UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     id_unidad_espacial UUID,  
4     tipo_elemento VARCHAR(100) NOT NULL,  
5     metodo_captura VARCHAR(100),  
6     precision_posicional NUMERIC(10,3),  
7     fuente_datos VARCHAR(255),  
8     observacion TEXT,  
9     geometria GEOMETRY(GEOMETRY, 9377) NOT NULL,  
10    fecha_levantamiento DATE,  
11    fecha_creacion TIMESTAMP DEFAULT NOW(),  
12  
13    CONSTRAINT fk_topografia_unidad_espacial  
14        FOREIGN KEY (id_unidad_espacial)  
15        REFERENCES ladm_sinic.unidad_espacial(id_unidad_espacial)  
16        ON DELETE SET NULL  
17 );
```

Listing 12: Tabla de topografía y representación

También se crea un índice espacial:

```
1 CREATE INDEX idx_topografia_geom  
2 ON ladm_sinic.topografia_representacion  
3 USING GIST (geometria);
```

Listing 13: Índice espacial para topografía y representación

9.6. Tabla de cartografía catastral

La tabla `cartografia_catastral` permite almacenar elementos cartográficos complementarios, como sectores, manzanas, zonas homogéneas, construcciones, barrios u otros objetos espaciales de referencia.

```
1 CREATE TABLE ladm_sinic.cartografia_catastral (  
2     id_cartografia UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     tipo_elemento VARCHAR(100) NOT NULL,  
4     codigo VARCHAR(100),  
5     nombre VARCHAR(255),  
6     fuente VARCHAR(255),  
7     escala VARCHAR(50),  
8     geometria GEOMETRY(GEOMETRY, 9377) NOT NULL,  
9     fecha_creacion TIMESTAMP DEFAULT NOW(),  
10    fecha_actualizacion TIMESTAMP DEFAULT NOW()  
11 );
```

Listing 14: Tabla de cartografía catastral

Índice espacial correspondiente:

```
1 CREATE INDEX idx_cartografia_geom  
2 ON ladm_sinic.cartografia_catastral  
3 USING GIST (geometria);
```

Listing 15: Índice espacial para cartografía catastral

10. Control de actualización mediante trigger

Con el fin de registrar automáticamente la fecha de modificación de los registros, se creó una función en el esquema auditoria. Esta función actualiza el campo `fecha_actualizacion` cada vez que se modifica un registro.

```
1 CREATE OR REPLACE FUNCTION auditoria.actualizar_fecha_modificacion()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.fecha_actualizacion = NOW();
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;
```

Listing 16: Función para actualización automática

La función se aplica sobre las tablas principales:

```
1 CREATE TRIGGER trg_update_interesado
2 BEFORE UPDATE ON ladm_sinic.interesado
3 FOR EACH ROW
4 EXECUTE FUNCTION auditoria.actualizar_fecha_modificacion();
5
6 CREATE TRIGGER trg_update_unidad_administrativa
7 BEFORE UPDATE ON ladm_sinic.unidad_administrativa
8 FOR EACH ROW
9 EXECUTE FUNCTION auditoria.actualizar_fecha_modificacion();
10
11 CREATE TRIGGER trg_update_unidad_espacial
12 BEFORE UPDATE ON ladm_sinic.unidad_espacial
13 FOR EACH ROW
14 EXECUTE FUNCTION auditoria.actualizar_fecha_modificacion();
15
16 CREATE TRIGGER trg_update_cartografia_catastral
17 BEFORE UPDATE ON ladm_sinic.cartografia_catastral
18 FOR EACH ROW
19 EXECUTE FUNCTION auditoria.actualizar_fecha_modificacion();
```

Listing 17: Triggers de actualización

11. Permisos finales

Una vez creadas las tablas, se asignan permisos al usuario de conexión del backend. Esto permite que la aplicación web pueda consultar, insertar, actualizar y eliminar registros sin requerir permisos de administrador.

```
1 GRANT SELECT, INSERT, UPDATE, DELETE
2 ON ALL TABLES IN SCHEMA ladm_sinic
3 TO sinic_app;
4
5 GRANT SELECT, INSERT, UPDATE, DELETE
6 ON ALL TABLES IN SCHEMA catalogos
7 TO sinic_app;
8
9 GRANT USAGE ON ALL SEQUENCES
10 IN SCHEMA ladm_sinic
11 TO sinic_app;
12
13 GRANT USAGE ON ALL SEQUENCES
14 IN SCHEMA catalogos
15 TO sinic_app;
```

Listing 18: Permisos finales para el backend

12. Verificación de la base de datos

Para comprobar que las tablas fueron creadas correctamente, se puede ejecutar la siguiente consulta:

```
1 SELECT table_schema, table_name
2 FROM information_schema.tables
3 WHERE table_schema IN ('ladm_sinic', 'catalogos', 'auditoria')
4 ORDER BY table_schema, table_name;
```

Listing 19: Consulta de verificación de tablas

El resultado esperado debe incluir las siguientes tablas principales:

- ladm_sinic.interesado
- ladm_sinic.unidad_administrativa
- ladm_sinic.derecho_interesado
- ladm_sinic.unidad_espacial
- ladm_sinic.topografia_representacion
- ladm_sinic.cartografia_catastral

También se puede comprobar el funcionamiento de PostGIS mediante:

```
1 SELECT ST_AsText(
2     ST_SetSRID(
3         ST_MakePoint(5000000, 2000000),
4         9377
5     )
6 );
```

Listing 20: Prueba de funcionamiento de PostGIS

13. Prueba de inserción espacial

Como prueba inicial, se puede insertar una unidad administrativa y posteriormente asociarle una geometría poligonal como unidad espacial.

```
1  INSERT INTO ladm_sinic.unidad_administrativa (  
2      codigo_predial,  
3      matricula_inmobiliaria,  
4      departamento,  
5      municipio,  
6      direccion,  
7      tipo_predio,  
8      destinacion_economica,  
9      area_terreno,  
10     area_construida,  
11     avaluo_catastral  
12 )  
13 VALUES (  
14     '110010100100100010001000000000',  
15     '50C-123456',  
16     'Bogota_D.C.',  
17     'Bogota_D.C.',  
18     'Calle_10_#_20-30',  
19     'Urbano',  
20     'Residencial',  
21     120.00,  
22     95.00,  
23     250000000  
24 );
```

Listing 21: Inserción de unidad administrativa de prueba

```

1  INSERT INTO ladm_sinic.unidad_espacial (
2      id_unidad_administrativa,
3      tipo_unidad_espacial,
4      etiqueta,
5      area_calculada,
6      geometria
7  )
8  SELECT
9      id_unidad_administrativa,
10     'Predio',
11     'Predio_de_prueba',
12     120.00,
13     ST_GeomFromText(
14         'POLYGON((5000000 2000000, 5000010 2000000, 5000010 2000012,
15             5000000 2000012, 5000000 2000000))',
16         9377
17     )
18 FROM ladm_sinic.unidad_administrativa
WHERE codigo_predial = '110010100100100010001000000000';

```

Listing 22: Inserción de unidad espacial de prueba

La consulta de validación permite revisar la relación entre la unidad administrativa y su geometría:

```

1  SELECT
2      ua.codigo_predial,
3      ua.direccion,
4      ue.tipo_unidad_espacial,
5      ST_Area(ue.geometria) AS area_geometrica,
6      ST_AsText(ue.geometria) AS geometria_wkt
7  FROM ladm_sinic.unidad_administrativa ua
8  JOIN ladm_sinic.unidad_espacial ue
9  ON ua.id_unidad_administrativa = ue.id_unidad_administrativa;

```

Listing 23: Consulta de validación espacial

14. Configuración para conexión futura del backend

La conexión del backend se realizará mediante variables de entorno. Para ello se propone un archivo `.env` con la siguiente estructura:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=bde_ladm_sinic
DB_USER=sinic_app
DB_PASSWORD= La contraseña establecida
```

Esto permitirá que el backend se conecte a la base de datos sin escribir directamente los datos de acceso dentro del código fuente.

““latex

15. Relación con el enfoque LADM_COL SINIC

La estructura propuesta se relaciona con el enfoque LADM_COL SINIC porque organiza la información catastral a partir de componentes esenciales del modelo: los interesados, las unidades administrativas, las unidades espaciales y los elementos de representación geográfica. Esta organización permite vincular la información jurídica, administrativa y espacial del predio dentro de una misma base de datos.

En el modelo propuesto, las tablas principales cumplen las siguientes funciones:

- **interesado**: almacena la información de los actores vinculados con los predios, como propietarios, poseedores, ocupantes u otras personas naturales o jurídicas.
- **unidad_administrativa**: representa el componente administrativo, jurídico y económico del predio, incluyendo datos como código predial, matrícula inmobiliaria, destino económico, áreas y avalúo catastral.
- **derecho_interesado**: permite relacionar los interesados con las unidades administrativas, indicando el tipo de derecho, el porcentaje de participación y la vigencia de dicha relación.
- **unidad_espacial**: contiene la representación geométrica del predio mediante geometrías tipo polígono, lo cual permite realizar consultas espaciales y asociar la información al territorio.
- **topografia_representacion**: almacena elementos espaciales relacionados con la representación física del predio, como vértices, puntos de control, mojones o linderos.

- `cartografia_catastral`: permite registrar elementos cartográficos complementarios, como sectores, manzanas, zonas, construcciones u otros objetos geográficos de referencia.

Aunque la base de datos no implementa la totalidad del estándar LADM_COL SINIC, sí conserva una estructura simplificada y funcional que permite representar los componentes básicos de la información catastral. Además, la inclusión de geometrías espaciales, relaciones entre tablas, campos de vigencia temporal y mecanismos de auditoría permite soportar el desarrollo académico de una aplicación web con operaciones CRUD y manejo de información geográfica. ““

16. Conclusiones

La preparación de la base de datos permitió definir una estructura inicial organizada, funcional y coherente con los requerimientos del proyecto. La decisión de utilizar PostgreSQL con PostGIS permite incorporar capacidades espaciales necesarias para la gestión de información catastral.

La separación mediante esquemas facilita la administración de los componentes principales del sistema, diferenciando las tablas del modelo, los catálogos y los elementos de auditoría. Asimismo, el uso de UUID como claves primarias mejora la identificación única de los registros y favorece una futura integración con servicios web.

La estructura propuesta permite soportar los CRUD exigidos para interesados, unidades administrativas, unidades espaciales, topografía y cartografía catastral. Además, el uso de geometrías con SRID 9377 permite orientar el almacenamiento espacial hacia un contexto colombiano.

Finalmente, esta fase constituye la base técnica para continuar con el desarrollo del backend y el frontend. Una vez validada la base de datos, el siguiente paso consiste en construir la API de conexión, definir las rutas CRUD y posteriormente desarrollar la interfaz web que permitirá interactuar con la información catastral.